Deploy the Nuxt application

Documentation index[1] · ภาษาไทย[2] · Deploy contracts[3] · Operations[4]

Reviewed against the repository on **28 October 2026** (Asia/Bangkok). This guide covers the existing Nuxt 4 application, its API and reconciliation worker. Blockchain purchases currently use **Polygon Amoy, chain ID 80002, and test tUSDT**. Hosting the application on a public domain does not make its contracts a mainnet deployment.

1. Deployment layout and prerequisites

```
Browser / connected wallet
          |
    HTTPS reverse proxy
          |
    Nuxt Node server ---- PostgreSQL
          |
    Polygon Amoy RPC ---- ReferralDistributor + AmoyTestUSDT
```

Prepare Node.js **24+**, npm, PostgreSQL **17+**, an existing database, a domain and HTTPS termination. Use a continuously running Node process: the application reconciles pending blockchain operations every **15 seconds**. Static hosting with `npm run generate` cannot run these APIs or the worker. A platform that suspends processes between requests needs a separate worker design that this repository does not provide.

Run commands from the repository root. The examples use Bash on Linux; on Windows PowerShell use `npm.cmd` in place of `npm`.

Keep the source and lockfile for the release. The repository's `compose.yaml` supplies a **development PostgreSQL service only**; it is not a production application deployment definition. Provision production database credentials and backups separately.

2. Install and configure

On the build/operator machine, install dependencies including the development tools needed to build and compile contracts:

```
npm ci --include=dev
test -f .env || cp .env.example .env
```

PowerShell equivalent for creating the configuration without replacing an existing file:

```
npm.cmd ci --include=dev
if (-not (Test-Path -LiteralPath .env)) { Copy-Item -LiteralPath .env.example -Destination .env }
```

Edit `.env` privately. This is a template; replace the example domain, database credentials and empty blockchain values before enabling purchases:

```
NODE_ENV=production
NITRO_HOST=127.0.0.1
NITRO_PORT=3000
DATABASE_URL=postgresql://APP_USER:APP_PASSWORD@DB_HOST:5432/APP_DATABASE
NUXT_PUBLIC_APP_URL=https://app.example.com
NUXT_PUBLIC_WALLET_MODE=external
NUXT_PUBLIC_PASSWORD_LOGIN_ENABLED=false

NUXT_EVM_RPC_URL=https://rpc-amoy.polygon.technology/
NUXT_EVM_CHAIN_ID=80002
NUXT_EVM_TOKEN_ADDRESS=
NUXT_EVM_EXPLORER_API_KEY=
NUXT_REFERRAL_PAYMENT_MODE=contract
NUXT_REFERRAL_CONTRACT_ADDRESS=
NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY=
NUXT_REFERRAL_MIN_CONFIRMATIONS=2
NUXT_REFERRAL_MIN_GAS_BALANCE=0.003
```

Use the database provider's SSL settings for hosted PostgreSQL, for example `?sslmode=verify-full` when supported with its certificate configuration. URL-encode special characters in database credentials. The database must already exist, and the application role needs permission to apply migrations as well as read/write application tables. Restrict database access to the application/operator network.

`NUXT_PUBLIC_APP_URL` must match the browser's exact origin: scheme, hostname and port when nonstandard. Redirect alternate domains to it. `NUXT_PUBLIC_*` settings are browser-visible; keys belong only in server configuration.

For contract payments, obtain the token/distributor addresses and matching authority key using the contract deployment guide[3]. An existing installation must retain its own addresses and signer. The application can start before these are configured, but purchases remain unavailable.

`NUXT_EVM_EXPLORER_API_KEY` is optional for startup and purchases; indexed wallet history needs a key with Amoy access. The generic transfer-evidence settings `NUXT_EVM_TREASURY_ADDRESS` and `NUXT_EVM_MIN_CONFIRMATIONS` are separate from the distributor's fee recipient; see the full environment reference[4].

3. Prepare the database and administrator

Confirm that `DATABASE_URL` selects the intended database, then run:

```
npm run db:status
npm run db:migrate
npm run admin:list
```

For a fresh installation with no administrator, run:

```
npm run admin:create
```

The command prompts for a **12–128 character** password and prints the assigned numeric member ID. Store these credentials securely and sign in at `/admin/login`. There are no default admin credentials. `LOOP_ADMIN_PASSWORD` is available for noninteractive setup; remove it from runtime configuration after setup.

Administrators keep using member ID/password login even when member password login is disabled. To manage contract fees, sign in as the administrator first and link the external wallet that owns the distributor. Application admin access alone does not grant contract ownership. See connected wallets[5] before linking an address already used by another account.

Startup also applies pending migrations from SQL bundled inside `.output`. Explicit migration before rollout makes failures easier to investigate. Preserve applied SQL files: the migration runner checks their names and checksums. Database reset is not a deployment step.

4. Build and start

Build a Node-server release, with development dependencies still installed:

```
npm run typecheck
npm run build
node --env-file=.env .output/server/index.mjs
```

On a build platform that auto-selects another deployment preset, explicitly set `NITRO_PRESET=node-server` before building. In Bash use `NITRO_PRESET=node-server npm run build`; in PowerShell use `$env:NITRO_PRESET = 'node-server'` followed by `npm.cmd run build`.

The deployable application artifact is the **entire `.output` directory**, including `server`, `public` and bundled migrations. Build on a compatible target platform. Operator commands such as migrations, administrator creation and contract deployment still need their source files and installed dependencies; they are not commands inside `.output`.

The production entry point does not load `.env` automatically. The command above explicitly loads it through Node. If hosting injects the environment, use `node .output/server/index.mjs` and supply `NODE_ENV=production` and all required variables through that host. Inherited environment variables take precedence over Node's env file; remove stale overrides when changing settings. See Nuxt's environment-file documentation.

The Node entry point and host/port variables follow Nitro's Node runtime documentation. Bind to `127.0.0.1` for a proxy on the same machine; use `0.0.0.0` when a container or hosting router needs to reach the process, with suitable network restrictions.

5. Keep the server running on Linux

This systemd example assumes a dedicated **existing** `unxo` service user, the release at `/opt/unxo/app`, and a private `.env` readable by that user. Replace paths and the Node executable with your installation's values (`command -v node`). A service does not inherit an interactive shell's Node version-manager setup.

Create `/etc/systemd/system/unxo.service`:

```
[Unit]
Description=UNXO Nuxt server
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=unxo
Group=unxo
WorkingDirectory=/opt/unxo/app
Environment=NODE_ENV=production
Environment=NITRO_HOST=127.0.0.1
Environment=NITRO_PORT=3000
ExecStart=/usr/bin/node --env-file=/opt/unxo/app/.env /opt/unxo/app/.output/server/index.mjs
Restart=always
RestartSec=5
NoNewPrivileges=true
UMask=0077

[Install]
WantedBy=multi-user.target
```

Limit `.env` permissions to the owner, ensuring that the service user can read it. Then enable the service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now unxo
sudo systemctl status unxo
sudo journalctl -u unxo -n 100 --no-pager
```

Keep `REFERRAL_DEPLOYER_PRIVATE_KEY` off the running application host when deployment is handled separately. Runtime still requires the original matching `NUXT_REFERRAL_AUTHORITY_PRIVATE_KEY`. Restart the service after changing runtime settings.

6. Domain, HTTPS and reverse proxy

Point DNS at the server and provision a valid TLS certificate using your hosting provider or certificate tooling. This Nginx example assumes the certificate files already exist; replace the domain and certificate paths before enabling it:

```
server {
    listen 80;
    server_name app.example.com;
    return 301 https://app.example.com$request_uri;
}

server {
    listen 443 ssl;
    server_name app.example.com;
    ssl_certificate /etc/letsencrypt/live/app.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/app.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_cache off;
    }
}
```

Validate and reload after saving the site configuration in your Nginx installation:

```
sudo nginx -t
sudo systemctl reload nginx
```

The proxy must preserve the public host and HTTPS scheme: origin validation and secure session cookies depend on them. The headers use Nginx's proxy directives. When a CDN terminates TLS before another proxy, configure that trusted proxy chain to retain the original scheme. Do not expose the backend port directly. Disable shared caching of `/api/*`; the application returns `Cache-Control: no-store`.

7. Verify the deployed application

From an external machine, check the actual public domain:

```
curl -fsS https://app.example.com/ -o /dev/null
curl -i https://app.example.com/api/auth/session
curl -fsS https://app.example.com/api/platform-settings
```

On Windows use `curl.exe`; for the first command replace `/dev/null` with `NUL`. There is no dedicated `/api/health` route. An anonymous session request should return HTTP 200 with `{"user":null}` and `Cache-Control: no-store`; platform settings exercise database reads. These checks do not prove that blockchain configuration is ready.

- Open the homepage in Thai and English and sign in at `/admin/login`.
- Connect a member wallet on Amoy and sign in through the canonical HTTPS domain; verify the session survives reload and its cookie has `Secure` and `HttpOnly`.
- Complete the contract checks[3], then use a test member with tUSDT and test POL for a level-1 purchase. Wait for confirmation and inspect purchase/accounting history.
- Confirm the service restarts successfully and pending transaction reconciliation continues. A missing explorer key should affect indexed history rather than prevent login.

8. Updates, recovery and common failures

Before updating, record the release revision and contract addresses. Back up PostgreSQL, private runtime configuration and `.data/contracts` on the operator machine. Finish pending transactions before changing contract addresses or payment modes.

Build the next release separately from the directory serving live traffic. Review migration compatibility, pause application writes for the switch when necessary, apply migrations, switch to the complete new artifact, restart and repeat the checks above. For the systemd layout, use `sudo systemctl restart unxo` after switching the release.

Keep the previous artifact for application rollback, but confirm that it supports the migrated schema before restoring it. There is no automatic down-migration. Restore a database backup only as an explicit recovery operation after reconciling chain activity: database restoration does not undo token transfers. Preserve the authority key across deployments rather than generating a new one on each release.

Symptom	Check
Startup fails	Node version, service paths/permissions, database connectivity and migration errors in service logs
HTTP 502	Node service running, listening host/port and proxy upstream
Wallet login returns 403/503	Exact `NUXT_PUBLIC_APP_URL`, forwarded host/scheme and stale process environment
Cookie or write request rejected	HTTPS, proxy headers, matching browser Origin and canonical domain
Contract unavailable	Amoy RPC/80002, correct token/distributor, version 2 and matching quote signer
Pending purchase	Preserve purchase ID/hash, inspect RPC/receipts/confirmations and allow reconciliation; avoid submitting a second payment blindly
Migration history/checksum mismatch	Restore the original applied SQL; create a new forward migration for changes

Source references: package scripts[6], runtime config[7], environment template[8], database startup[9], reconciliation worker[10], authentication[11].

Repository references

Numbered references identify repository files listed below. Include the source repository when sharing documents that require those files. External website links remain clickable in the PDF.

1. `docs/README.md`
2. `docs/th/deploy-nuxt.md`
3. `docs/en/deploy-contracts.md`
4. `docs/en/operations.md`
5. `docs/connected-wallets.md`
6. `package.json`
7. `nuxt.config.ts`
8. `.env.example`
9. `server/plugins/database.ts`
10. `server/plugins/referral-reconciliation.ts`
11. `server/utils/auth.ts`